

Advanced Lesson 16 - Exercises:

Packages

Mikey Ferguson

MATH 7013 - Advanced Mathematical Modeling

1: Loading ComputerArithmetic`

```
Needs["ComputerArithmetic`"]
```

Needs loads a package if it is not already loaded and does nothing if it is, which makes it safe to re-evaluate. The older idiom is Get, written with a double angle bracket, but Get reloads unconditionally and re-running it produces shadowing warnings.

ComputerArithmetic` is one of the legacy standard add-on packages. It is still shipped with this installation - the file sits under AddOns/Packages/ComputerArithmetic/Kernel/init.m inside the Wolfram Engine layout - so the exercise works as written. It is worth confirming rather than assuming: several of the packages this era of the course drew on have since been removed or folded into the kernel.

2: Packages on the current context path

```
$ContextPath
```

```
Length[$Packages]
```

```
Sort[Select[$Packages, StringFreeQ[#1, "`" ~~ __ ~~ "`"] &]]
```

\$ContextPath now has "ComputerArithmetic`" prepended, which is what makes the package's exported names usable without qualification.

\$Packages is the much longer list of every context loaded in this session, whether or not it is currently on the search path - the distinction being

between what exists and what is visible. In this version it runs to well over a hundred entries, most of them internal subcontexts of the chat and machine-learning subsystems, so the third cell filters out the nested ones and sorts what remains. `ComputerArithmetic`` appears in that filtered list, which is the confirmation Part 1 was after.

3: The primary functions in `ComputerArithmetic``

`Names["ComputerArithmetic`*"]`

The package exports seventeen names: `Arithmetic`, `ComputerNumber`, `ExponentRange`, `IdealDivide`, `IdealDivision`, `MachineError`, `MicroscopePlot`, `MicroscopicErrorPlot`, `MixedMode`, `NaN`, `RoundingRule`, `RoundToEven`, `RoundToInfinity`, `SetArithmetic`, `Truncation`, `Ulp` and `Ulp`s.

The package models finite-precision arithmetic explicitly: `SetArithmetic` fixes a number of digits and a base, `ComputerNumber` constructs a number in that system, `Ulp` measures the gap between representable neighbours, and `MachineError` reports the discrepancy between the exact value and the representable one. It is the practical counterpart to Lesson 2, which studied precision from the language's side rather than the hardware's.

`?ComputerArithmetic`*`

The wildcard form of `"?"` prints the usage message for every name in the context at once, which is the quickest way to see what a small package offers.

4: Information on `MicroscopicErrorPlot`

`?MicroscopicErrorPlot`

`Information["ComputerArithmetic`MicroscopicErrorPlot", LongForm → False]`

What there is to find. The usage message gives two signatures.

`MicroscopicErrorPlot[f, {x, a}]` plots the error incurred by using machine arithmetic to evaluate the expression `f` in a small neighbourhood of `a`, and

`MicroscopicErrorPlot[f, {x, a, n}]` plots that error from `a - n ulps` to `a + n ulps`. An ulp is a unit in the last place: the gap between one representable machine number and the next, which the same package's `Ulp` function measures.

Beyond the usage string, `Information` reports that the symbol has attributes `HoldFirst` and `Protected` - `HoldFirst` because the expression must not be evaluated before the package can control the arithmetic under which it is evaluated - and one option, `Joined`, defaulting to `True`. Documentation is

reported as None: there is no Documentation Center page and no examples. That is the whole answer to "what information there is on it". Legacy add-on packages were documented in the printed Technical Report that shipped with them, not in the online help system, so reading the package source at the path FindFile reports below is the remaining option, and for a package of this vintage it is often the only real documentation available.

```
FindFile["ComputerArithmetic`"]
```